SELLER-ONLY COMMERCIALISATION SYSTEM / VERSION 1.0

SHIPSAFE AI

COMMERCIAL LAUNCH MASTER

Your AI-built app working is not proof that it is ready to launch.

Brand messaging, long-form sales copy, pricing, diagnostic funnel, objection handling, seven-day launch campaign, social content, visual copy, email and direct-message assets, and seller operations.

PRIVATE SELLER ASSET

Do not include this master document inside either customer edition. Copy only the customer-facing text required for publication.

Prepared 13 June 2026 | Founding campaign: 13-20 June 2026

How to use this master

This document separates the permanent product position from the temporary June 2026 acquisition campaign. The product remains evergreen. The Google/Kaggle course is used only as a timely path into the market.

- Keep the primary category and promise consistent across every channel.
- Change the entry problem to match the audience: broken code, security uncertainty, capstone submission, rollback, authorization, or production readiness.
- Lead with a useful diagnosis or technical insight. Present the product after the buyer recognizes the gap.
- Never imply guaranteed security, legal compliance, automatic remediation, or a result that has not been independently verified.
- Do not bundle implementation support, custom reviews, or lifetime updates into the price unless intentionally added later.
- Use the Solo and Commercial licence distinctions exactly; do not blur personal project use and client-service use.

COMMERCIAL PRINCIPLE

Sell certainty about what has and has not been verified - not certainty that the software can never fail.

Contents

1. Brand and category architecture

Category

Launch assurance for AI-built applications

One-line description

A production guardrail, verification, security, and recovery system for applications built with AI coding agents.

Core promise

PROMISE

Know whether your AI-built app is genuinely safe, functional, and ready to launch - then fix what is not.

Primary market statement

Your AI-built app working is not proof that it is ready to launch.

Supporting statement

Stop guessing. Verify the code, permissions, data, tests, recovery path, and launch evidence before real users expose the gaps.

The market enemy

Fluent false confidence: an application that looks finished while critical controls remain unverified.

The SHIPSAFE Verification Loop

The SHIPSAFE Verification Loop: Constrain -> Inspect -> Test -> Secure -> Recover -> Prove -> Launch.

1. Constrain the agent and the task.

2. Inspect the actual implementation and dependencies.

3. Test normal, failure, and cross-user paths.

4. Secure data, secrets, inputs, tools, and agent permissions.

5. Recover through known-good checkpoints and compatible rollback.

6. Prove completion with independently reviewable evidence.

7. Launch only after an explicit go/no-go decision.

Voice and visual language

Use	Avoid
Precise, Authoritative, Calm, Protective, Evidence-led, Anti-hype	fearmongering, guaranteed-security claims, generic AI imagery, developer gatekeeping, vague “best practice” filler, unlimited support promises

Message hierarchy

Category message

Your AI-built app working is not proof that it is ready to launch.

Problem message

Generated software can conceal false success, missing authorization, exposed configuration, regressions, and untested recovery.

Mechanism message

SHIPSAFE turns launch confidence into a verification loop with persistent controls and evidence.

Product message

The buyer receives project files, tests, security checks, recovery procedures, a scorecard, and release evidence templates.

Action message

Install the guardrails, run the tests, resolve unknowns, and make the launch decision.

2. Buyer, problem, and value architecture

Primary buyer

A nontechnical or lightly technical builder who already has an AI-built application, agent, capstone, internal tool, or client project and cannot independently determine whether it is safe, correct, recoverable, and ready for users.

Secondary buyer

A developer, freelancer, consultant, studio, or agency that wants a persistent control layer and repeatable pre-launch audit workflow around coding agents.

Who it is for

- Nontechnical and lightly technical founders using AI coding agents to ship products.
- Developers who want persistent control files and evidence requirements around agentic coding.
- Google/Kaggle course participants completing an AI-agent capstone during June 15-19, 2026.
- Freelancers and consultants who need a repeatable readiness audit before handing client projects over.
- Builders using Claude Code, Codex, Gemini, Cursor, Supabase, or comparable agentic development workflows.

Who it is not for

- Anyone seeking a security guarantee, penetration test, legal opinion, or compliance certification.
- Teams unwilling to inspect evidence or make launch decisions themselves.
- Buyers wanting a one-click tool that automatically fixes every codebase.
- People looking for a generic prompt dump or a “make money overnight” guide.

The invisible failures SHIPSAFE surfaces

The interface works, but the data is mocked.

A generated fallback, fixture, or static response can make a broken integration appear functional.

Authentication works, but authorization does not.

A user may be signed in and still be able to retrieve another user’s records by changing an identifier.

The secret is outside .env, but still inside the browser bundle.

Public client code, build-time variables, logs, or source maps can expose credentials and private configuration.

The agent “fixed” the error by hiding it.

Broad exception handling, empty fallbacks, removed validation, or suppressed errors can convert a visible failure into false success.

A working feature was changed during an unrelated task.

Without protected files, regression checks, and a minimal-diff rule, local improvement can create remote damage.

The repository can be rolled back, but the database cannot.

Code recovery does not reverse destructive or incompatible data migrations.

Admin controls are hidden, not protected.

A missing button is not server-side authorization.

The app passed the developer’s session, not a clean-user test.

Cookies, cached permissions, local state, and privileged accounts can conceal first-user failures.

Value ladder

Layer	Buyer value
Visible value	A complete set of editable and ready-to-use files.
Functional value	Fewer uncontrolled changes, stronger tests, explicit recovery, and a structured launch decision.
Economic value	Avoided rework, incident response, exposed customer data, failed delivery, and emergency rebuilds.
Emotional value	The builder knows what remains unknown instead of relying on the agent’s confidence.
Commercial value	The Commercial buyer can turn the framework into a repeatable paid readiness audit.

3. Offer and pricing structure

PRICING RULE

Use a one-time purchase. Do not add a subscription merely to create recurring revenue. The product is designed to be self-contained and low-obligation.

SHIPSAFE Solo

Founding price	Standard price	Designed for
US$39	US$59	A founder, learner, operator, or independent builder launching their own applications.

Included

Finished operating manual, quick start, agent guardrails, project-control templates, testing and security checks, checkpoint and rollback procedures, readiness scorecard, capstone assets, and bonus quick tools.

Licence position

Use for projects owned and operated by the purchaser. No client-service use, resale, redistribution, sublicensing, or template sharing.

SHIPSAFE Commercial

Founding price	Standard price	Designed for
US$129	US$179	A freelancer, consultant, studio, or agency applying the system inside paid client work.

Included

Everything in Solo, plus editable premium source documents, client intake, audit-report assets, acceptance templates, and commercial project-use rights.

Licence position

Use internally across client engagements and deliver completed reports or adapted outputs to clients. The underlying files, templates, and system may not be resold, redistributed, sublicensed, or offered as a competing template product.

Price-change language

Founding pricing is available during the initial June 13-20, 2026 campaign. After the campaign, Solo moves to US$59 and Commercial moves to US$179. Use an honest date-based change. Do not reset or fabricate scarcity.

What is deliberately excluded

- Custom implementation
- Code review or penetration testing
- Legal or compliance certification
- Guaranteed launch approval
- Unlimited support
- Lifetime updates
- Resale or redistribution rights
- A hosted scanner or source-code upload service

Edition comparison

Capability	Solo	Commercial
Finished manuals and Quick Start	Included	Included
Agent guardrails and project controls	Included	Included
Security, testing, rollback, and scorecard	Included	Included
Capstone and portfolio pack	Included	Included
Use on purchaser-owned projects	Included	Included
Use inside paid client work	Not included	Included
Editable premium source documents	Not included	Included
Client intake and audit-report assets	Not included	Included
Resale or redistribution	Not permitted	Not permitted

4. Long-form sales-page copy

USE

This is publication-ready source copy. Adapt formatting and button labels to the seller's chosen environment without weakening the claims boundary.

Hero

Your AI-built app working is not proof that it is ready to launch.

Stop guessing. Verify the code, permissions, data, tests, recovery path, and launch evidence before real users expose the gaps.

SHIPSAFE AI gives builders the exact guardrails, tests, security checks, rollback procedures, and evidence system required to move from "it seems to work" to an informed launch decision.

PRIMARY CTA

Get SHIPSAFE AI

SECONDARY CTA

See what it checks

TRUST LINE

Local files. No source-code upload required. No subscription. No false guarantee.

The Problem

AI coding agents can create complete-looking applications quickly. They can also modify multiple files, run commands, invent plausible implementation details, hide failed integrations behind fallbacks, and declare a task complete without independently proving the result.

That creates a new category of launch risk: fluent false confidence.

The application may look finished while its permissions, data boundaries, secrets, failure paths, tests, recovery procedures, or production configuration remain unverified.

The Cost Of Guessing

A broken button is visible. A missing authorization check is not.

A syntax error stops the build. Mocked success can pass a demonstration.

A failed deployment is obvious. A database migration with no recovery path may appear successful until rollback is needed.

SHIPSAFE AI is designed for the invisible failures - the ones that survive a superficial demo.

The Mechanism

The SHIPSAFE Verification Loop: Constrain -> Inspect -> Test -> Secure -> Recover -> Prove -> Launch.

1. Constrain: define scope, protected files, decision gates, and what the agent may change.

2. Inspect: inventory generated code, dependencies, data paths, permissions, mocks, and external integrations.

3. Test: verify primary journeys, failure modes, clean-user behavior, and regression protection.

4. Secure: test authorization, secrets, inputs, storage, RLS, agent permissions, and prompt-injection boundaries.

5. Recover: create known-good checkpoints, version migrations, and prove the rollback procedure.

6. Prove: collect changed files, commands, test output, security results, and acceptance-criteria evidence.

7. Launch: make an explicit go/no-go decision using the readiness score, critical controls, and residual-risk record.

What Is Included

Agent guardrails

Persistent instructions for Claude Code, Codex, Gemini, Cursor, and other coding agents so working code, protected files, tests, and high-risk decisions are not casually overwritten.

Scope and change control

Project context, scope lock, protected-file list, change plan, acceptance criteria, decision log, and definition of done.

Testing and failure verification

Manual QA, regression test matrix, failure-mode tests, release evidence, and a requirement that completion claims be supported by proof.

Security preflight

Authorization, secrets, prompt-injection, input, storage, RLS, dependency, and data-leakage checks.

Checkpoints and recovery

Known-good checkpoints, rollback planning, recovery context collection, and an emergency recovery runbook.

Launch-readiness scoring

A weighted spreadsheet with 68 controls, category scores, critical-failure tracking, and an evidence-based go/no-go decision.

Capstone and portfolio pack

Project planner, submission checklist, README, demo script, and case-study structure.

Commercial audit system

Client intake, readiness audit, report structure, and deliverable acceptance tools for paid client work.

The Readiness Scorecard

The included spreadsheet evaluates 68 launch controls across scope, recovery, functional correctness, authentication, data privacy, secrets, AI safety, observability, deployment, and agent evidence.

Unknown controls score zero until verified. Critical failures remain visible. Category scores reveal exactly where launch confidence is weakest.

The output is not a decorative badge. It is an evidence-led decision: Launch Ready, Conditional Launch, Staging Ready, High Risk, or Stop - Not Ready.

Who It Is For

Nontechnical and lightly technical founders using AI coding agents to ship products.

Developers who want persistent control files and evidence requirements around agentic coding.

Google/Kaggle course participants completing an AI-agent capstone during June 15-19, 2026.

Freelancers and consultants who need a repeatable readiness audit before handing client projects over.

Builders using Claude Code, Codex, Gemini, Cursor, Supabase, or comparable agentic development workflows.

Who It Is Not For

Anyone seeking a security guarantee, penetration test, legal opinion, or compliance certification.

Teams unwilling to inspect evidence or make launch decisions themselves.

Buyers wanting a one-click tool that automatically fixes every codebase.

People looking for a generic prompt dump or a “make money overnight” guide.

Why It Is Different

Not a prompt bundle. The files become persistent project controls.

Not a generic checklist. The controls connect to evidence, scoring, rollback, and release decisions.

Not an automated-security promise. The product clearly separates verification from guarantee.

Not tied to one platform. Dedicated and universal files support multiple coding-agent workflows.

Not support-dependent. The buyer receives a self-contained system with a defined operating sequence.

How To Use It

1. Open the three-page Quick Start.
2. Copy the appropriate agent instruction file into the project.
3. Complete Project Context, Scope Lock, Protected Files, and Acceptance Criteria.
4. Create a known-good checkpoint.
5. Run the Launch Readiness Scorecard and authorization/regression tests.
6. Resolve or explicitly accept remaining risk.
7. Complete the release evidence packet and go/no-go decision.

Pricing

SHIPSAFE Solo - US$39 founding launch price; US$59 standard price.

SHIPSAFE Commercial - US$129 founding launch price; US$179 standard price.

One-time purchase. No subscription. No implementation service included. Licence terms apply.

Final Cta

Do not ask an AI coding agent whether its own work is safe to launch.

Give the project boundaries. Demand evidence. Test the invisible paths. Preserve a recovery point. Make the launch decision yourself.

CTA

Get SHIPSAFE AI

MICROCOPY

Download the complete edition immediately after purchase. Start with the Quick Start. Keep the master files private.

5. Edition, checkout, and delivery copy

Solo Title

SHIPSAFE AI - Solo Edition

Solo Short

Production guardrails, security checks, testing controls, rollback procedures, and a 68-control launch-readiness scorecard for your own AI-built projects.

Commercial Title

SHIPSAFE AI - Commercial Edition

Commercial Short

The complete SHIPSAFE system plus editable source files, client audit assets, and commercial project-use rights for freelancers, consultants, studios, and agencies.

Order Bump

Optional future add-on idea: SHIPSAFE Vertical Test Packs. Do not include or promise this until it exists.

Confirmation

Your files are ready. Download the ZIP, keep the original archive unchanged, and begin with 00_START_HERE/SHIPSAFE_AI_Quick_Start.pdf.

Licence Note

By purchasing, you receive the licence stated inside your edition. You do not receive ownership of the SHIPSAFE AI brand, templates, or underlying system.

Refund Position

Because this is an immediately delivered digital product, define the refund policy clearly before sale and apply the consumer-law requirements relevant to the seller and buyer. Do not copy an absolute “no refunds” statement without legal review.

Compact edition cards

SHIPSAFE SOLO

US$39 founding price. A founder, learner, operator, or independent builder launching their own applications. Finished operating manual, quick start, agent guardrails, project-control templates, testing and security checks, checkpoint and rollback procedures, readiness scorecard, capstone assets, and bonus quick tools.

SHIPSAFE COMMERCIAL

US$129 founding price. A freelancer, consultant, studio, or agency applying the system inside paid client work. Everything in Solo, plus editable premium source documents, client intake, audit-report assets, acceptance templates, and commercial project-use rights.

Post-purchase start message

Your SHIPSAFE files are ready. Download the archive and keep the original ZIP unchanged as your master copy. Open 00_START_HERE/SHIPSAFE_AI_Quick_Start.pdf first. Then install the correct agent guardrail file, complete project context and scope controls, create a known-good checkpoint, and run the Launch Readiness Scorecard. Do not launch while any Critical control is failed or unknown.

6. Free diagnostic funnel

Headline

Is your AI-built app actually ready to launch?

Complete the 12-point test. No source-code upload required. Every unresolved Critical item should be treated as a launch blocker until verified.

Question set

1. Can a normal user retrieve or modify another user’s record by changing an ID, URL, query, or request body? **[Critical]**

2. Have you tested the product from a clean account or private browser session? **[High]**

3. Can you identify every mock, fixture, placeholder, fallback, or hardcoded success response? **[Critical]**

4. Has every public/client-side bundle been checked for secrets and private configuration? **[Critical]**

5. Does the coding agent report every changed file, command, test, and unresolved assumption? **[High]**

6. Are protected features covered by regression tests before unrelated changes are accepted? **[Critical]**

7. Can you restore the last known-good application version and the compatible database state? **[Critical]**

8. Do failed integrations produce visible failure rather than invented, empty, or static success? **[Critical]**

9. Are admin and privileged actions enforced server-side rather than merely hidden in the interface? **[Critical]**

10. Are uploads, URLs, webhooks, forms, and AI-retrieved content treated as untrusted input? **[High]**

11. Are development, staging, and production configuration separated? **[High]**

12. Is there a written launch/no-launch decision supported by test and security evidence? **[High]**

Result copy

0-2 unresolved: Near launch review

Your project may be close, but any Critical unresolved item still blocks a responsible launch decision. Complete the evidence packet and re-test the affected controls.

3-5 unresolved: Material launch risk

Several important controls remain unverified. The product may work while data, authorization, failure, or recovery gaps remain hidden.

6-8 unresolved: High launch risk

Do not rely on the demonstration as evidence. Establish boundaries, checkpoints, clean-user tests, and a structured security preflight before launch.

9-12 unresolved: Stop - not ready

The project lacks enough verified control to justify public launch. Preserve the current state, install guardrails, and complete the full readiness workflow.

Conversion block

RESOLVE THE FAILED CONTROLS

SHIPSAFE AI gives you persistent agent guardrails, testing and security controls, checkpoint and rollback procedures, release evidence, and the complete 68-control readiness scorecard.

Diagnostic disclaimer

This test is an educational screening tool. It is not a security guarantee, penetration test, legal opinion, compliance certification, or substitute for qualified review of high-risk systems.

7. Objections and frequently asked questions

Objection handling

“My app already works.”

Working in one session proves only that one path produced an acceptable result once. It does not verify authorization, data isolation, failure behavior, secret handling, regression protection, recovery, or a clean-user journey.

“My AI tool already tests its work.”

The same system that generated the implementation should not be the only authority deciding whether it is complete. SHIPSAFE requires independently reviewable evidence and explicit launch criteria.

“I can ask ChatGPT for a checklist.”

A generated checklist is not a persistent operating system. SHIPSAFE connects agent boundaries, project scope, security tests, regression protection, recovery, evidence, scoring, and licensing into one reusable workflow.

“I can use free security scanners.”

Automated scanners are useful but narrow. They do not confirm business logic, mock data, acceptance criteria, authorization journeys, agent scope, rollback readiness, or whether a failure was converted into fake success.

“This sounds too technical.”

The product is specifically designed to translate technical launch risk into structured questions, pass/fail evidence, and a clear go/no-go decision.

“I will deal with it after launch.”

After launch, every unresolved issue is exposed to real users, real data, public reputation, and production dependencies. The cheapest time to create a checkpoint and prove access boundaries is before release.

“I only built a small MVP.”

Small scope lowers complexity; it does not eliminate authentication, data, credential, failure, billing, or recovery risk.

Frequently asked questions

Is SHIPSAFE AI a security guarantee?

No. It is an engineering control and verification framework. It reduces avoidable uncertainty and exposes missing evidence, but it is not a penetration test, compliance certification, legal opinion, or guarantee against defects or attacks.

Do I need to be a developer?

No. The Quick Start, scorecard, checklists, and agent files are designed to make technical verification legible to nontechnical builders. Some remediation work may still require qualified technical help.

Which coding agents does it support?

The pack includes dedicated files for Claude Code, Codex-style AGENTS.md workflows, Gemini, and Cursor, plus universal guardrails that can be adapted to other agents.

Does it scan my private code?

No. The product is delivered as local files and templates. Buyers decide what to share with their tools. The 12-point fast test and readiness scorecard can be completed without uploading source code to SHIPSAFE.

Will it fix my application automatically?

No. It tells the builder and their coding agent what must be verified, what evidence is missing, where launch risk exists, and how to control, test, and recover the work.

What is the difference between Solo and Commercial?

Solo is for projects owned and operated by the purchaser. Commercial adds client-use rights, editable premium source materials, and client audit assets for paid engagements.

Can I resell the templates?

No. Neither licence permits resale, redistribution, sublicensing, white-labelling of the underlying system, or offering the files as a competing template product.

Can I use Commercial for multiple clients?

Yes, inside paid client work, subject to the included licence. You may deliver completed reports and project-specific outputs, but not the reusable source system itself.

Is this only useful during the Google/Kaggle course?

No. The course provides a timely acquisition angle. The controls are evergreen for any AI-built app, agent, micro-SaaS, internal tool, or client project.

What happens after purchase?

The buyer downloads the edition they purchased, opens the Quick Start, installs the appropriate guardrail files, completes project context and scope controls, runs the readiness assessment, resolves failures, and records the go/no-go decision.

Are updates included?

The purchase covers the version delivered. Do not promise lifetime updates or implementation support unless you intentionally add those terms later.

Why is “Unknown” treated as zero in the scorecard?

Because an unverified control is not evidence of safety or correctness. The framework rewards proof, not assumption.

8. Seven-day launch campaign

Campaign objective: establish the category, distribute useful diagnosis and proof, intercept the June 15-19 AI-agent course attention, and transition into evergreen production-readiness positioning on June 20.

Saturday, June 13 - Category creation: working is not ready

Goal

Introduce the invisible-risk category and publish the product without sounding like another AI template launch.

Flagship post

Your AI-built app working is not proof that it is ready to launch.

A generated application can load, accept input, and produce a clean demo while still:

- returning mocked data when the real integration fails
- allowing one user to request another user's records
- shipping private configuration inside the browser bundle
- suppressing errors until every failure looks like success
- changing a working feature during an unrelated fix
- leaving no compatible rollback path for the database

The dangerous failures are not always the ones that crash. They are the ones that survive the demonstration.

I built SHIPSAFE AI to give AI-assisted builders persistent agent guardrails, security and authorization tests, regression controls, recovery procedures, and a 68-control launch-readiness decision system.

The rule is simple: an agent's statement that the task is complete is not evidence.

Changed files, commands, tests, security results, acceptance criteria, and a recovery point are evidence.

Call to action

Run the free 12-point fast test or review SHIPSAFE AI.

Proof asset

Show the scorecard dashboard and one agent guardrail file.

Sunday, June 14 - Protect the project before the first major agent session

Goal

Reach people preparing for the Google/Kaggle course and anyone starting a new AI-built app.

Flagship post

Before an AI coding agent writes the next feature, give the project five things:

1. PROJECT_CONTEXT.md - what the product is, who it serves, and how the system works.
2. SCOPE_LOCK.md - what is in scope, out of scope, and forbidden without approval.
3. PROTECTED_FILES.md - the files and behaviors that must not be casually replaced.
4. ACCEPTANCE_CRITERIA.md - the evidence required before a task can be called complete.
5. A known-good checkpoint - the exact state you can return to when the “small fix” becomes a rewrite.

“Be careful” is not a control system. Persistent boundaries are.

Call to action

Use the free setup list; the complete files are inside SHIPSAFE AI.

Proof asset

Share a redacted view of the five-file project-control layer.

Monday, June 15 - Course day one: capstone boundaries

Goal

Intercept active course attention with a genuinely useful setup protocol.

Flagship post

Starting an AI-agent capstone today? Decide this before you build:

- the one user outcome the demo must prove
- what the agent may change without asking
- which files, features, and data paths are protected
- what counts as a real integration rather than a mock
- which tests must pass before the next task begins
- how you will restore the last working version

A capstone becomes easier when the scope is smaller and the evidence is stronger.

Do not use the final day to discover that the agent built a convincing interface around an unverified backend.

Call to action

Use the Capstone Planner and Submission Checklist inside SHIPSAFE AI.

Proof asset

Show one completed example criterion, not product screenshots only.

Tuesday, June 16 - False success

Goal

Make the buyer recognize a failure pattern they cannot solve with a generic prompt.

Flagship post

Four ways an AI coding agent can make a broken application look functional:

1. Catch the exception and return an empty response.
2. Replace the failed API call with static sample data.
3. Remove the validation causing the test to fail.
4. Display a success state before the durable operation completes.

The interface works. The product does not.

A proper release check forces the system to reveal failure, prove the real integration, preserve validation, and verify durable state after the action.

Call to action

Use the Failure Mode Tests and Release Evidence Packet.

Proof asset

A before/after example of fake success versus verified failure.

Wednesday, June 17 - Authorization and data boundaries

Goal

Connect launch risk directly to user trust without making unsupported security guarantees.

Flagship post

Authentication answers: “Who are you?”
Authorization answers: “Are you allowed to do this to this record?”

An AI-built app can have a polished login page and still fail the second question.

Before launch, test with two ordinary accounts:

- Can User A request User B’s record by changing an ID?
- Can User A update or delete it?
- Can a normal user call an admin endpoint directly?
- Can a private file be guessed or opened with another account?
- Does suspension or deletion invalidate the old session?

Do not test permissions only from the developer account.

Call to action

Use the 12-case Authorization Test Matrix.

Proof asset

Show the matrix columns: actor, action, expected, actual, evidence.

Thursday, June 18 - Rollback before repair

Goal

Reach builders currently experiencing agent-caused regressions.

Flagship post

Before asking an AI coding agent to “fix everything,” create a recovery boundary.

Record:

- the current branch and commit
- the last known-good tag
- uncommitted changes
- current environment configuration
- migration state
- the command that verifies core behavior
- the exact files the next task may change

Then isolate the fix.

A code rollback is not enough when the database, storage, or external configuration changed too. Recovery must restore a compatible system, not merely an older folder.

Call to action

Use the Checkpoint Protocol and Recovery Runbook.

Proof asset

Demonstrate the included checkpoint script on a harmless sample repository.

Friday, June 19 - Submission and launch evidence

Goal

Convert course participants and builders preparing a public release.

Flagship post

Before submitting or publishing an AI-built app, ask for evidence - not confidence.

Your release packet should contain:

- every changed file
- commands run
- test output
- authorization results
- secret and dependency checks
- acceptance-criteria results
- known limitations
- rollback evidence
- the final go/no-go decision

A polished demo explains what the product does. A release packet proves what was actually verified.

Call to action

Complete the SHIPSAFE Launch Readiness Scorecard before submission.

Proof asset

Show a blurred/fictional release evidence packet and readiness decision.

Saturday, June 20 - From capstone or prototype to product

Goal

Transition from event urgency into the evergreen offer.

Flagship post

The capstone is finished. The product work begins.

Before turning a prototype into a portfolio asset, client deliverable, or micro-SaaS:

- remove mock and privileged development paths
- verify clean-account onboarding
- test authorization across two users
- separate production configuration
- define monitoring and cost ceilings
- prove rollback
- record known limitations
- make an explicit launch decision

SHIPSAFE AI turns those tasks into one repeatable operating system rather than another round of improvised prompts.

Call to action

Founding launch pricing closes; standard pricing begins after the campaign.

Proof asset

Solo vs Commercial comparison and the complete file inventory.

9. Short-form content bank

Use these as standalone posts, replies, carousel captions, video hooks, or supporting lines under product screenshots. Do not post them all at once.

Post 1

An AI agent saying “done” is a completion claim. Test output, changed files, security results, and acceptance criteria are completion evidence.

Post 2

A page loading proves rendering. It does not prove authorization, durable state, or a real integration.

Post 3

The most dangerous AI coding failure may be a fallback that looks exactly like success.

Post 4

Do not let the agent delete a failing test to make the test suite green.

Post 5

Hide an admin button if you want. Protect the admin action on the server if you need security.

Post 6

Your .env file can be private while your browser bundle still exposes the value.

Post 7

A Git rollback does not automatically reverse a database migration.

Post 8

Test your app with two ordinary accounts, not only one privileged developer account.

Post 9

The smaller the capstone scope, the stronger the evidence you can collect.

Post 10

“Make the app production-ready” is not an acceptance criterion.

Post 11

Every high-risk task should begin with a known-good checkpoint and end with a changed-file report.

Post 12

If failure returns empty data and a success message, the interface is lying on behalf of the system.

Post 13

Unknown is not pass. In SHIPSAFE, an unverified control scores zero until evidence exists.

Post 14

A coding agent should stop when requirements conflict. Inventing the missing decision is not autonomy; it is uncontrolled scope.

Post 15

If you cannot restore the previous compatible state, you do not have a tested rollback plan.

Post 16

Mock data is useful in development. Undisclosed mock data is dangerous in a launch decision.

Post 17

Client-side validation improves experience. Server-side validation protects the system.

Post 18

A security checklist without an owner, evidence, and a go/no-go decision becomes decoration.

Post 19

The app can be technically impressive and still be operationally unlaunchable.

Post 20

SHIPSAFE AI is not a security guarantee. It is the system that stops “we assumed” from being treated as “we verified.”

Post 21

The correct time to document recovery is before the incident, not while the agent is still editing.

Post 22

One vague fix request can cross authentication, database, UI, and infrastructure boundaries. Scope it before execution.

Post 23

A clean demo is marketing evidence. A regression suite is engineering evidence.

Post 24

The objective is not zero risk. It is visible risk, verified controls, and an informed launch decision.

Post 25

Before launch, search the codebase for mock, demo, fixture, fallback, TODO, FIXME, hardcoded, and placeholder.

Post 26

A completed task report should include what changed, what ran, what passed, what failed, and what remains unknown.

Post 27

The agent should make the smallest viable change, not the most impressive rewrite.

Post 28

Production readiness is not a vibe. It is a collection of verified controls.

10. Email and direct-message copy

USE RESPONSIBLY

Do not mass-message strangers. Use email where consent or an existing relationship exists. Use direct messages as permission-based follow-up to a relevant public conversation.

Email sequence

Launch announcement

Subject: Your AI-built app working is not proof it is ready

AI coding agents are excellent at producing complete-looking applications. They are less reliable as the only authority deciding whether those applications are safe, correct, recoverable, and ready for real users.

SHIPSAFE AI is a complete launch-assurance system for AI-built applications. It gives you persistent agent guardrails, scope controls, security and authorization tests, regression checks, checkpoints, rollback procedures, release evidence, and a weighted readiness scorecard.

It is not a prompt dump, security guarantee, or implementation service. It is the operating system that turns “the agent says it is done” into a decision supported by evidence.

Founding launch pricing:
- Solo: US$39
- Commercial: US$129

Start with the free 12-point fast test, or get the complete edition.

False success education

Subject: The failure that still looks like a successful demo

A generated application does not need to crash to be broken.

It can catch an exception, return an empty response, switch to static data, remove the failing validation, and continue displaying a success state.

That is why SHIPSAFE includes dedicated failure-mode tests and a release evidence packet. The goal is to force failures to become visible and require proof that the real integration completed.

A polished interface is useful. It is not release evidence.

Authorization education

Subject: A login page does not protect another user’s data

Authentication proves identity. Authorization decides whether that identity may read, update, delete, download, or administer a specific resource.

Before launch, create two ordinary accounts and deliberately attempt cross-user access. Test the API and storage directly, not only the visible interface.

SHIPSAFE includes a 12-case authorization matrix and evidence fields so the result is recorded rather than assumed.

Commercial licence

Subject: For consultants auditing AI-built client projects

The Commercial Edition is designed for freelancers, consultants, studios, and agencies that need a repeatable pre-launch control system inside paid client work.

It includes the complete Solo product plus editable premium source files, client intake, audit-report assets, deliverable acceptance, and commercial project-use rights.

You may use the framework across client engagements and deliver completed, project-specific outputs. You may not redistribute or resell the underlying SHIPSAFE system.

Launch close

Subject: Founding SHIPSAFE pricing closes today

Founding launch pricing closes today.

SHIPSAFE Solo moves from US$39 to US$59.
SHIPSAFE Commercial moves from US$129 to US$179.

The product remains the same: a self-contained system for constraining coding agents, testing invisible failure paths, verifying permissions and secrets, preserving recovery points, collecting release evidence, and making an informed launch decision.

No subscription. No implementation service. No unsupported security guarantee.

Direct-message bank

Relevant reply - broken code

That failure pattern is exactly why I now isolate high-risk changes behind a known-good checkpoint, protected-file list, and explicit regression criteria. I put the free version of that process into a 12-point readiness test; happy to share the public link where community rules allow.

Relevant reply - security uncertainty

A useful first check is to test with two ordinary accounts and attempt cross-user read, update, delete, admin, and private-file access. I built a structured authorization matrix for this inside SHIPSAFE; the short test can be completed without uploading code.

Permission-based direct message

You mentioned you are preparing an AI-built app for launch. I created a structured readiness system covering agent boundaries, authorization, secrets, regression, rollback, and release evidence. May I send you the short 12-point test?

After permission

Here is the fast test. Treat every Critical unresolved item as a launch blocker until verified. The full SHIPSAFE pack contains the agent files, test matrices, recovery runbook, and weighted scorecard if you need the complete system.

Commercial prospect

I saw that your work includes client applications built with AI coding agents. SHIPSAFE Commercial is a reusable pre-launch control and audit system with client intake, editable reports, authorization tests, rollback procedures, and commercial project-use rights. It is designed to produce a client-specific evidence packet rather than resell templates.

No-pressure close

This will not automatically secure or fix a codebase, and it is not a penetration test. It is useful when you need a repeatable way to expose unknowns, control agent changes, record evidence, and make a defensible launch decision.

11. Visual copy and product proof

Visual direction

- Dark navy technical foundation
- White or pale-slate content surfaces
- Teal for verified/control states
- Amber for unknown or conditional states
- Red only for failed or critical controls
- No robots, neon brains, fake code rain, padlock clichés, or speculative cyber imagery
- Show real product artifacts: scorecard, guardrail files, matrices, checklists, and recovery procedures

Ten launch cards

Category card

YOUR AI-BUILT APP WORKING IS NOT PROOF THAT IT IS READY TO LAUNCH.

Subline: Test the invisible paths before real users do.

False success card

THE INTERFACE WORKS. THE PRODUCT DOES NOT.

Mock data. Silent fallbacks. Suppressed failures. Premature success states.

Evidence card

“DONE” IS A CLAIM. EVIDENCE IS A RELEASE DECISION.

Changed files + commands + tests + security results + acceptance criteria + rollback.

Authorization card

A LOGIN PAGE DOES NOT PROTECT ANOTHER USER’S DATA.

Test two ordinary accounts. Try cross-user read, update, delete, admin, and file access.

Recovery card

A GIT ROLLBACK MAY NOT RESTORE YOUR DATABASE.

Recover a compatible system, not only an older code folder.

Scorecard card

68 CONTROLS. 10 CONTROL AREAS. ONE GO/NO-GO DECISION.

Unknown scores zero until evidence exists.

Product stack card

CONSTRAIN. INSPECT. TEST. SECURE. RECOVER. PROVE. LAUNCH.

The SHIPSAFE Verification Loop.

Solo card

SHIPSAFE SOLO

For builders launching their own AI-built projects. Founding price: US$39.

Commercial card

SHIPSAFE COMMERCIAL

Client audit assets + editable source + commercial project-use rights. Founding price: US$129.

Closing card

PRODUCTION READINESS IS NOT A VIBE.

It is a collection of verified controls.

Minimum proof set

- Quick Start cover and first-use sequence
- Operating Manual contents page
- AGENTS.md or universal guardrail excerpt
- Scorecard dashboard showing empty/unknown state and a completed fictional example
- Authorization test matrix
- Checkpoint protocol and recovery runbook
- Solo versus Commercial inventory comparison
- Commercial client audit template
- ZIP file inventory and checksum evidence
- Short sample-repository demonstration showing checkpoint -> constrained task -> tests -> changed-file report

Demonstration script

1. Open a harmless sample repository with one protected feature and one failing test.

2. Show PROJECT_CONTEXT, SCOPE_LOCK, PROTECTED_FILES, and ACCEPTANCE_CRITERIA.

3. Create a known-good checkpoint.

4. Give the agent a narrowly scoped task.

5. Show the resulting diff, commands run, test output, and task-completion report.

6. Run the relevant readiness controls and show how Unknown becomes Pass only after evidence is recorded.

7. End with the go/no-go decision; do not claim the sample proves universal security.

12. Seller operations and measurement

Launch checklist

☐ **Product integrity:** Confirm Solo and Commercial ZIPs open and match their licence scope.

☐ **Product integrity:** Retain an untouched seller-master archive and SHA-256 checksums.

☐ **Product integrity:** Confirm every customer-facing file uses the correct product version.

☐ **Claims:** Remove any wording that implies guaranteed security, compliance, zero risk, or automatic remediation.

☐ **Claims:** Keep the distinction between engineering framework and professional security/legal review visible.

☐ **Pricing:** Set the founding and standard prices deliberately; avoid fake perpetual countdowns.

☐ **Pricing:** Do not promise lifetime updates, unlimited support, or custom implementation unless operationally funded.

☐ **Licence:** Display the correct Solo or Commercial rights before purchase and include the licence inside delivery.

☐ **Proof:** Capture product screenshots: Quick Start, scorecard dashboard, agent guardrail, authorization matrix, recovery runbook, and file inventory.

☐ **Proof:** Record one short demonstration using a harmless sample repository.

☐ **Diagnostic:** Publish the 12-point fast test and score bands.

☐ **Content:** Prepare the seven flagship posts and daily short posts before launch begins.

☐ **Content:** Convert each flagship post into one carousel, one short video script, and three concise variants.

☐ **Community:** Answer existing questions substantively; do not mass-post links or manufacture testimonials.

☐ **Customer experience:** Ensure the post-purchase message tells buyers exactly which file to open first.

☐ **Customer experience:** Keep customer support boundaries explicit: product questions versus implementation work.

☐ **Legal/admin:** Complete name/trademark conflict review and jurisdiction-specific digital-product, consumer-law, privacy, tax, and refund review.

☐ **Measurement:** Track qualified views, fast-test starts, completions, product views, edition selected, and purchases using the seller’s chosen system.

☐ **Iteration:** Record every objection and confusion point, then improve copy without changing the product promise.

Signal hierarchy

Signal	Meaning	Action
Fast-test completion	Buyer recognizes the problem strongly enough to finish diagnosis.	Improve result-page relevance and product bridge.
Commercial edition views	Buyer sees client-service or agency value.	Strengthen audit proof and licence clarity.
FAQ expansion	A recurring objection is blocking purchase.	Answer it directly in main copy.
Shares/saves	Educational content has durable utility.	Create additional assets around the same failure pattern.
Qualified replies	The content crossed the correct builders.	Respond substantively and collect exact language.
Purchases	Offer, proof, price, and timing aligned.	Preserve the winning message before testing alternatives.

Content repurposing rule

Each flagship idea should produce: one long educational post, one 5-8 card carousel, one 30-60 second demonstration or narration, three short posts, one useful community answer, and one product-proof image. The idea stays consistent; only the format changes.

Support boundary

Customer support may clarify file location, licence scope, or intended use. It does not include debugging a buyer’s application, reviewing private code, confirming compliance, operating production systems, or guaranteeing launch approval. Any future implementation service should be separately scoped and priced.

13. Claims, licensing, and risk boundaries

Approved claim patterns

- Helps expose missing launch evidence.
- Provides persistent coding-agent guardrails.
- Structures security, authorization, testing, and recovery checks.
- Supports an informed go/no-go decision.
- Reduces avoidable uncertainty and uncontrolled agent changes.
- Can be used without uploading source code to SHIPSAFE.
- Includes a weighted 68-control readiness scorecard.

Do not claim

- Guaranteed secure, hack-proof, compliant, production-safe, or error-free.
- Automatically finds or fixes every vulnerability.
- Replaces a penetration test, legal review, security professional, or compliance audit.
- Guarantees approval, revenue, course success, deployment success, or zero incidents.
- Works identically for every stack without adaptation.
- Includes support, updates, or services not actually delivered.

Licence summary

Solo permits use on the purchaser's own projects. Commercial permits internal use across paid client engagements and delivery of completed project-specific outputs. Neither permits redistribution, resale, sublicensing, white-labelling of the underlying system, or creation of a competing template product. The included licence files remain controlling.

Brand and legal preflight

Before public commercial release, complete a name and trademark conflict check, and obtain appropriate jurisdiction-specific review for consumer law, digital-product terms, privacy, tax, refund treatment, disclaimers, and commercial licensing. SHIPSAFE AI remains a working product name until that review is complete.

14. Market timing and official sources

The launch timing uses current official signals. These sources support the temporary campaign angle and the underlying need for bounded, reviewable agentic workflows. They do not prove sales or guarantee demand.

Google and Kaggle AI Agents Intensive: Vibe Coding

Official confirmation of the June 15-19, 2026 course, production-ready agent focus, capstone, and prior course reach.

https://blog.google/innovation-and-ai/technology/developers-tools/kaggle-genai-intensive-course-vibe-coding-june-2026/

OpenAI - Codex for every role, tool, and workflow

Official June 2026 market context: more than five million weekly users and fast growth among non-developers.

https://openai.com/index/codex-for-every-role-tool-workflow/

Anthropic - Beyond permission prompts: making Claude Code more secure and autonomous

Official explanation of filesystem/network isolation, permission fatigue, prompt injection, and bounded agent access.

https://www.anthropic.com/engineering/claude-code-sandboxing

Anthropic - How we contain Claude across products

Official May 2026 discussion of unexpected agent behavior and containment.

https://www.anthropic.com/engineering/how-we-contain-claude

OpenAI Codex repository

Official description of Codex CLI as a local coding agent and supported editor workflows.

https://github.com/openai/codex

Permanent versus temporary positioning

PERMANENT

Launch assurance for any AI-built application, regardless of course, coding agent, or event.

TEMPORARY

A June 13-20 campaign that speaks directly to people building an AI-agent capstone during the Google/Kaggle course.

Final commercial standard

Production readiness is not a vibe. It is a collection of verified controls.

SHIPSAFE AI should be marketed with the same discipline it asks buyers to apply to software: precise scope, visible evidence, honest limitations, reversible decisions, and no unsupported claims.